



www.questpond.com

Contents

1. Explain your project architecture?	2
2. Architecture style VS Architecture pattern VS Design pattern.....	4
3. What are design patterns?.....	4
4. Which are the different types of design patterns?	5
5. Which design pattern have you used in your project?	6
6. Explain Singleton Pattern and the use of the same?	7
7. How did you implement singleton pattern?	7
8. Can we use Static class rather than using a private constructor?.....	8
9. Static vs Singleton pattern?.....	8
10. How did you implement thread safety in Singleton?.....	8
11. What is double null check in Singleton?	8
12. Can Singleton pattern code be made easy with Lazy keyword?	9
13. Can we rid of this double null check code?	10
14. What are GUI architecture patterns, can you name some?	10
15. Explain term Separation of concerns (SOC) ?.....	10
16. Explain MVC Architecture Pattern?.....	11
17. Explain MVP Architecture pattern?.....	11
18. What is the importance of interface in MVP ?.....	12
19. What is passive view?	12
20. Explain MVVM architecture pattern?	12
21. What is the difference between MVP and MVVM ?.....	13
22. What is a ViewModel?	13
23. When to use what MVP / MVC / MVVM?.....	13
24. MVC vs MVP vs MVVM?	14
25. Layered architecture vs Tiered?	14
26. What is SOLID?	15
27. What is the full form of SOLID?.....	15
28. What is the goal of SOLID ?.....	15
29. Question :- Explain SRP with a example ?.....	15
30. Question :- What is the benefit of SRP ?.....	15



www.questpond.com

31. Question :- Explain OCP with a example ?	15
32. Question :- What is the benefit of OCP ?	15
33. Question :- Can you explain LISKOV Principle and its violation ?	15
34. Question :- How can we fix LISKOV Problem ?	15
35. Question :- Explain Interface Segregation Principle?	15
36. Question :- Is there a connection between LISKOV and ISP ?	15
37. Question :- Define dependency inversion?	15
38. Question :- What is higher level module and lower level module ?	15
39. Question :- How does dependency inversion benefit , show with an example ?	15
40. Question :- Will only Dependency inversion solve decoupling problem ?	15
41. Question :- Why do developers move object creation outside high level module?	15
42. Question :- Explain IOC (Inversion of control) ?	15
43. Question :- Explain Dependency Injection with an example?	15
44. Question :- Is SOLID, IOC and DI design pattern or principle?	16
45. Question :- Is only SOLID Enough for good code/architecture ?	16
46. 2 or 3 Lessons on Design pattern	16
47. 1 o 2 lecture on DDD and Microservice architecture ?	16
48. Requirement to Architecture how did you go about it	16
49. How do you about writing a technical document?	16
50. Which UML diagrams have you used in your tech document	16
51. List of design patterns and Architecture patterns	16
GOF design patterns	16
Creational design patterns	16
Structural design patterns	17
Behavioral design patterns	17
Non-GOF Patterns	18

Explain your project architecture?

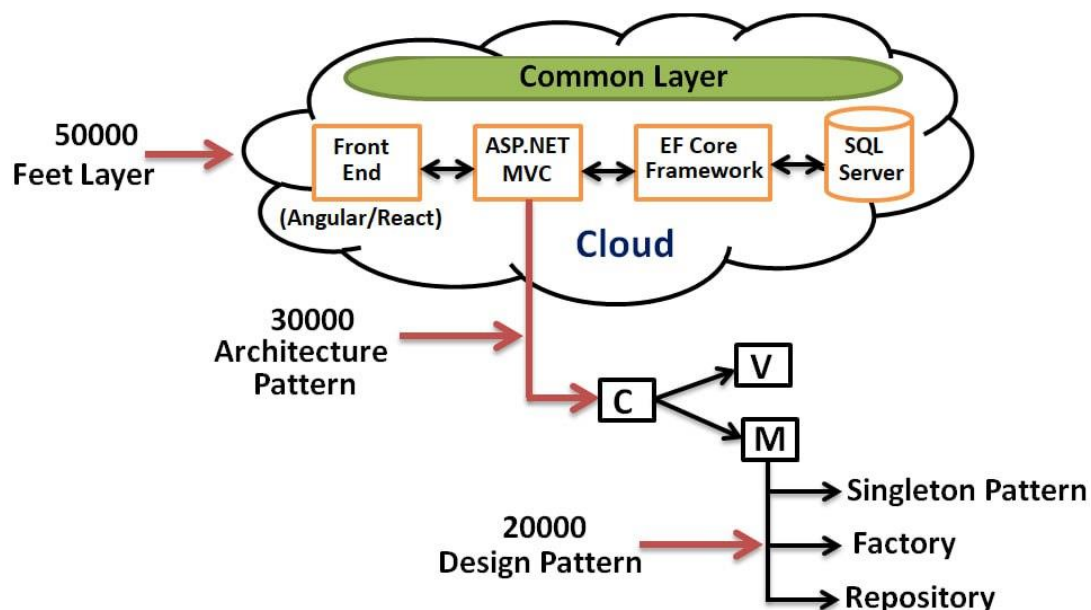


www.questpond.com

*Note: - For this question do not answer ONE LINERS like MVC, 3 Layer. Try to give a detailed answer. One special note **my architecture cannot be your architecture**, so **my answer can not be your answer**. So, try to see your architecture and express accordingly.*

So, the way to answer this question is to start from 50000 level go to 30000 level and land to 10000 level.

1. **The top layers:** - It's a 4-layer architecture with front end in Angular / React, Middle Layer in ASP.NET MVC, Data layer is EF and Database is SQL Server.
2. **Architecture patterns:** - Then ZOOM on the architecture patterns of the layers. So like the ASP.NET layer follows MVC architecture where in the model has business logic, view has the UI controls / look and feel and the controller connects the model with the view. If your front end is angular you can talk about MVC (Model view component) architecture.
3. **Design patterns:** - At the last level talk about any design patterns you have used. For example, you can say data access layer uses repository /UOW, factory pattern for centralizing object creation and so on.



It's a 4-layer architecture with front end in Angular, Middle Layer in MVC, Data access layer uses EF and DB in SQL Server.

ASP.NET follows MVC architecture where model has business logic, view look and feel and controller connect the model and the view.

In projects we have implemented best practices and design patterns like repository / UOW / Factory, iterator pattern and so on.



www.questpond.com

Architecture style VS Architecture pattern VS Design pattern

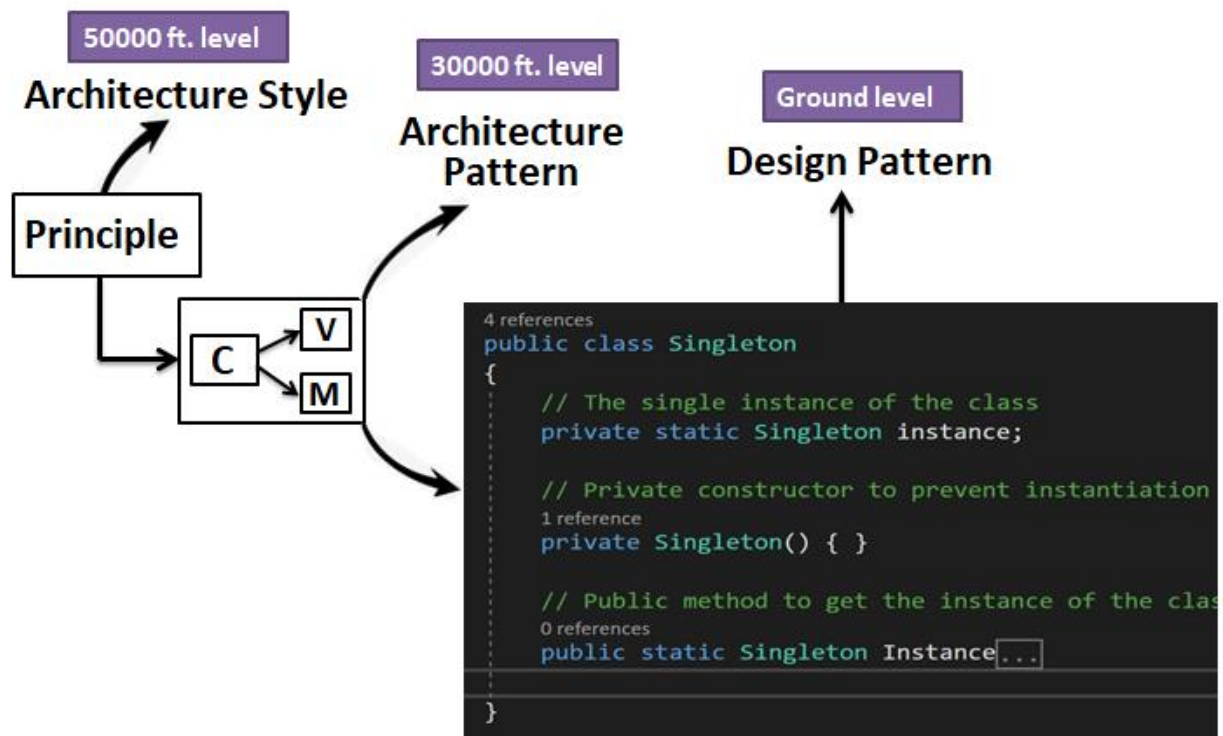
Note :- One of the important things during architect interview is to avoid vocabulary confusion. If you have vocabulary confusion the interviewer can lose confidence.

An architecture style represents a set of principles or guidelines that shape the overall structure and organization of a software system. Some examples are REST, SOA and so on.

So, for example, REST provides general principle on how to design networked application. Like its should be stateless, resource abstraction, manipulation of resource through representation and standard uniform interfaces.

Architecture pattern provides general structures / layers for designing software system. Some examples are MVC, Layered architecture, MVVM and so on.

Design pattern is solution for recurring problems in software design. And it has a very clear context. It operates at a very lower level, almost at the code level.



What are design patterns?



www.questpond.com

Design patterns are time tested solution for recurring architecture problems.

Also, with the above definition try to give one or two design patterns as examples. While giving examples remember three things: -

1. Do not give example of Singleton pattern so that you can stand out in the crowd.
2. Give one example of non-GOF pattern like repository, CQRS and so on.
3. And whatever patterns you say in example make sure you are fully aware of the same.

Examples of pattern please tailor as per your knowledge

“Adapter pattern” helps to make incompatible interfaces compatible.

“Iterator pattern” helps iterate over elements of an aggregate object sequentially without exposing the underlying representation of the object.

“Template pattern” define a skeleton of the algorithm in parent class and let subclass override specific steps.

“Repository pattern” helps to abstract and centralize the data access logic in an application. It provides a layer of separation between the application's business logic and the data access code, making the code more modular, maintainable, and testable.

Which are the different types of design patterns?

There are 3 categories of design pattern: -

- Creational: - Problems and solutions around object creation issues.
 - Prototype - A fully initialized instance to be copied or cloned
 - Singleton - A class of which only a single instance can exist
- Behavioral: - Problems and solutions around Communication between objects.
 - Chain of responsibility - A way of passing a request between a chain of objects
 - Command - Encapsulate a command request as an object
 - Mediator - Defines simplified communication between classes
 - Memento - Capture and restore an object's internal state
 - Template method - Defer the exact steps of an algorithm to a subclass
- Structural: - Solving concerns around class structure and object composition.
 - Adapter - Match interfaces of different classes
 - Composite - A tree structure of simple and composite objects
 - Decorator - Add responsibilities to objects dynamically



www.questpond.com

Which design pattern have you used in your project?

Note: - first and foremost do not answer SINGLETON pattern as it will not help you stand in the crowd. and remember: -

- My choice of patterns is not yours so pick what you are comfortable.
- Only talk about patterns you are confident of. Many patterns you have already used in your projects. If you can have closer look on them you can speak not only confidently but also naturally. When answers are natural interviewers love it.
- Do not just talk GOF but also talk about Non-GOF patterns.
- Yes, avoid singleton stand in the crowd. Its over used and abused in interviews.

Some of the most used patterns are: -

Most of the applications are CRUD so repository pattern comes at the top.

1. Repository pattern: - Acts like an abstract layer between Models and Data access technologies like EF, ADO.NET and so on. Data access logic is centralized making code maintainable, testable and modular.

With repository pattern, UOW goes like hand in gloves.

2. UOW (Unit of work): - This pattern helps to manage transactions and changes made to objects. It goes with repository pattern well.

We all know we use FOR EACH so much so the next pattern is also very must used. One important point to note here talk about IEnumerator and IEnumerable as they implement iterator by default.

3. Iterator pattern: - helps iterate over elements of an aggregate object sequentially without exposing the underlying representation of the object.
4. Factory pattern :- From creational pattern this is the most used one. Creates an instance of several derived classes.

Used when third party components are used.

5. Adapter pattern :- Makes incompatible interfaces compatible.
6. Decorator pattern :- Add behavior dynamically.
7. Command pattern:- Treat command as objects. Heavy use in CQRS
8. Façade :- Represent subsystem in a simplified way.
9. Composite - A tree structure of simple and composite objects
10. Template method - Defer the exact steps of an algorithm to a subclass



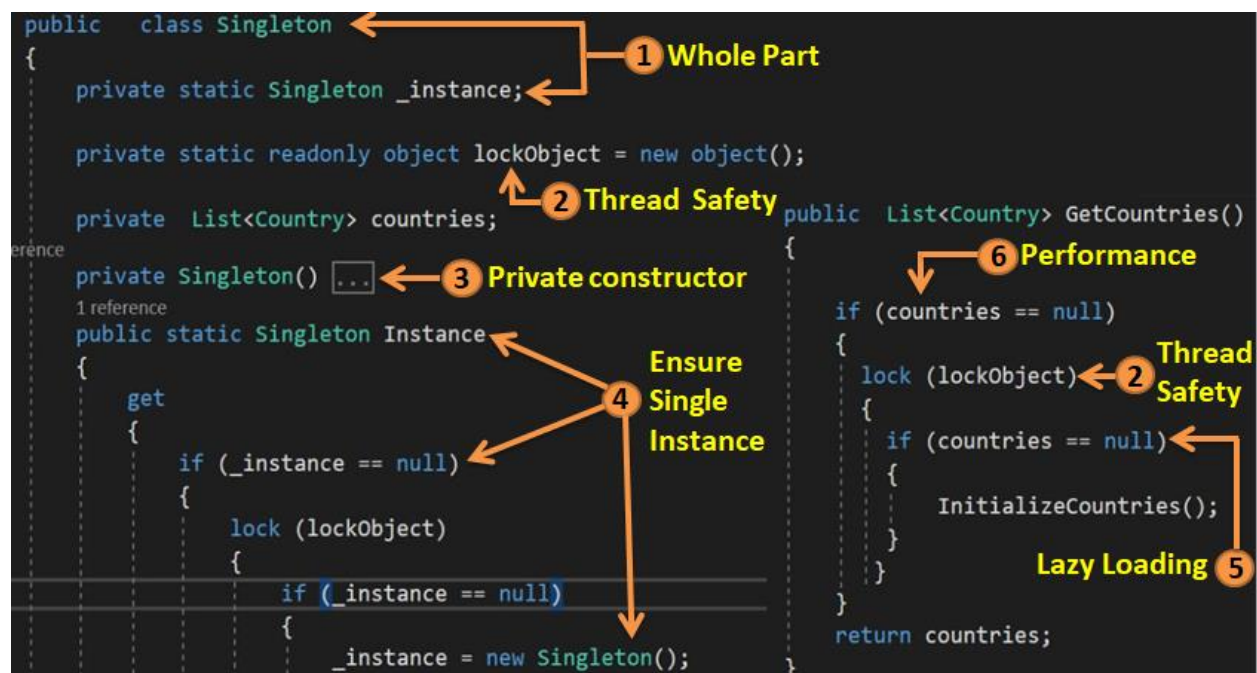
www.questpond.com

Explain Singleton Pattern and the use of the same?

Singleton pattern helps to create a single instance of an object. Some of the uses of Singleton patterns are: -

- Caching of data like Countries, States, Currencies and so on.
- Global sharing of data like common themes , hit counters and so on.

How did you implement singleton pattern?



To implement singleton pattern (Check the number with the above figure): -

1. Whole part relationship: - The first thing needed in singleton is a root class through which all shared objects should be exposed. This root class instance should be created inside the class and it should be static.
2. Thread safety: - Whenever we are loading the singleton object use the "lock" keyword to make sure only one thread manipulates at a time.
3. Private constructor: - Make sure the root class can not be instantiated from outside.
4. Ensure root object is single instance:- use the double null check and make sure that only one instance is created and also lock is not executed unnecessarily.
5. Lazy loading implementation:- We would like to load the objects when demanded then loading it unnecessarily.
6. Performance for threading: - NULL check before the locking to ensure that we do not execute lo



www.questpond.com

Can we use Static class rather than using a private constructor?

Static vs Singleton pattern?

Both the above question answered.

- Keyword VS Design Pattern: - Static is a language keyword while Singleton is a Design pattern.
- Loose many OOP features: - If you make a class static you cannot implement interfaces, cannot inherit and so on.
- Lazy loading / Thread Safety: - When you make a class static, object is created in the first call itself without giving you option of when to load and when not. Second you should also make sure its thread safe as it's a global object.

How did you implement thread safety in Singleton?

C# "lock" keyword helps to implement thread safety.

Whatever code is in the scope of "lock" keyword will get executed by only ONE thread at time avoiding any thread unsafe situation.

```
1 reference
public static Singleton Instance // 4. Giving access
{
    get
    {
        if (_instance == null)
        {
            lock (lockObject)
            {
                if (_instance == null)
                {
                    _instance = new Singleton();
                }
            }
        }
        return _instance;
    }
}
2 reference
```

Thread Safety

What is double null check in Singleton?

Double null check is done for two purposes.

Internal null check: - This is done to make sure the instance load only once and its loaded OnDemand.

External null check :- This is done so that "lock" is not acquired unnecessarily. "lock" is an intensive process and should be executed only when the singleton is null.

```
1 reference
public static Singleton Instance // 4. Giving access
{
    get
    {
        if (_instance == null)
        {
            lock (lockObject)
            {
                if (_instance == null)
                {
                    _instance = new Singleton();
                }
            }
        }
        return _instance;
    }
}
```

Performance (points to the first `if` statement)

Double Null Check (points to the second `if` statement inside the lock)

Performance Intensive (points to the `lock` statement)

Lazy loading (points to the `_instance = new Singleton();` line)

Can Singleton pattern code be made easy with Lazy keyword?

Yes, by using LAZY keyword we can make the code size smaller. LAZY makes code thread safe and also does late initialization.

```
public List<Country> GetCountries()
{
    // 6. Lock performance
    if (countries == null)
    {
        lock (lockObject) // 2. Thread
        {
            if (countries == null) // 5.
            {
                InitializeCountries();
            }
        }
    }
    return countries;
}
```

11 Lines of code (points to the first code block)

```
public List<Country> GetCountries()
{
    return countries.Value;
}
```

Compacted to Single line of code (points to the second code block)



www.questpond.com

Can we rid of this double null check code?

Yes by using LAZY keyword you can get rid of double null check.

What are GUI architecture patterns, can you name some?

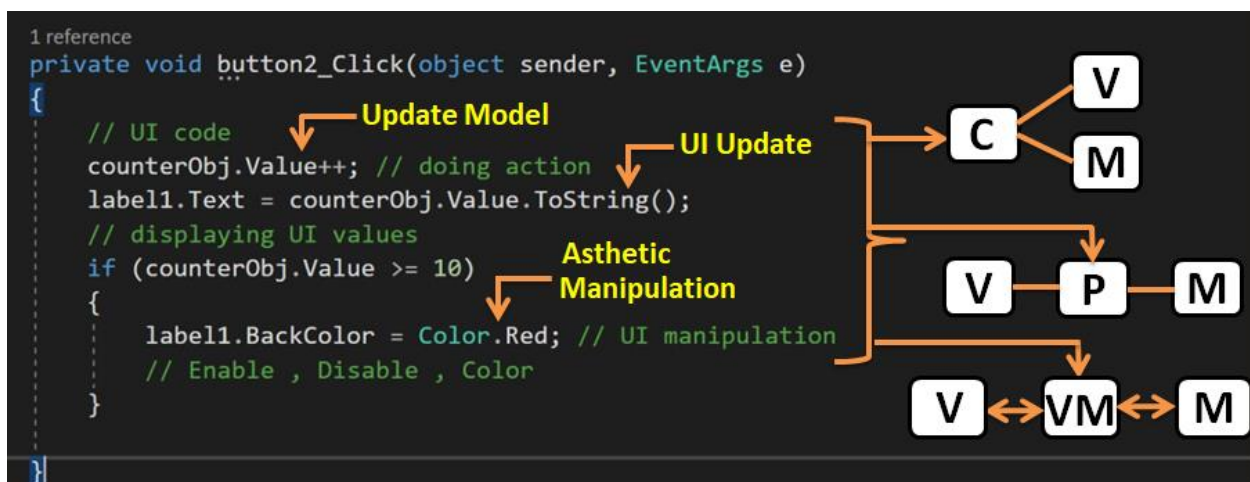
GUI Architecture pattern refers to how to organize structurally the interaction between User, View (UI) and components (Models, Business logic) in a software application. Its helps to maintain clear separation of concerns in distinct layers.

Some of the GUI architecture patterns are: -

MVC: - Model View Controller.

MVP: - Model View Presenter.

MVVM: - Model View ViewModel.



If you take an example of a typical GUI code it has lot of plumbing code as shown in the below figure. This makes the code more complex and difficult to maintain. A typical UI code does following things:-

- Calling, loading and updating model.
- Updating UI from the Model and vice-versa.
- Enabling, Disabling, changing color and so on.

So rather than GUI getting loaded with all CONCERNS we put them in to different sections and layers.

Explain term Separation of concerns (SOC) ?

Separation of Concerns (SoC) is a design principle that advocates breaking a software system into distinct sections, each addressing a separate concern or aspect of functionality.



www.questpond.com

For example MVC , MVP and MVVM apply separation of concerns concept.

Explain MVC Architecture Pattern?

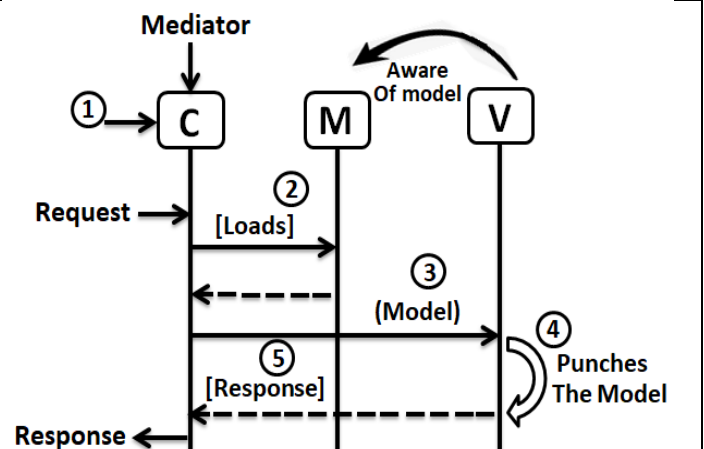
In MVC architecture we divide the project in to three primary layers: Controller, View and Model.

Controller: - Takes user inputs, loads model and passes to the view.

Model: - This section has business logic.

View: - This section handles the look and feel, positioning and alignment etc.

In MVC the first hit comes to the controller, controller then loads the model and attaches it to the view. Controller acts like a mediator between View and Model.



Explain MVP Architecture pattern?

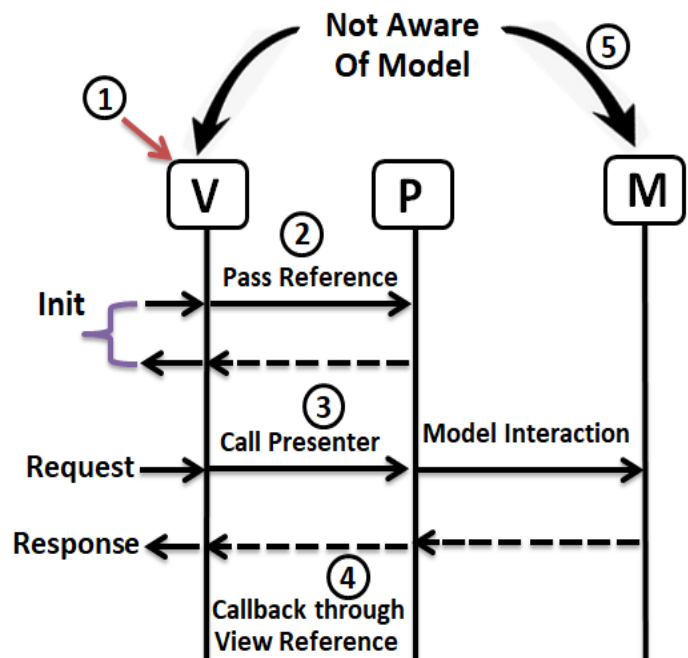
In MVP architecture we divide the project in to three primary layers: Presenter, View and Model.

View: - This section takes user control, handles the look and feel, positioning and alignment etc. It forwards all user interaction to the presenter.

Presenter: - Takes the input from the view, loads the model and updates the view accordingly.

Model: - This section has business logic.

In MVP the first hit comes to the view, view passes the call to presenter, presenter calls the model and updates the view. Presenter updates the view via a callback.





www.questpond.com

What is the importance of interface in MVP ?

Interface helps to connect the view and presenter. Presenter calls / Updates the View via the Interface callback.



What is passive view?

Passive view is view which does not contain any logic related to user interaction and presentation. Passive View submits itself to the presenter for all UI interaction logic and follows the presenter like deity. Here view does two things: -

- Has User controls.
- Passes the UI interaction to the presenter.
- Waits for the presenter call back to update the UI.

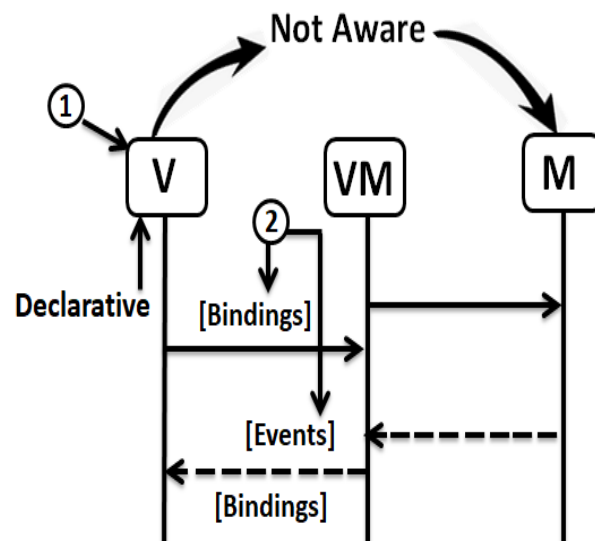
Explain MVVM architecture pattern?

In MVM architecture we divide the project in to three primary layers: ViewModel, View and Model.

View: - This section takes user control, handles the look and feel, positioning and alignment etc. All Actions and updates to the view happens through **automated bindings**.

ViewModel: - This layer is the mediator between View and Model and does all heavy lifting. It talks with the model , updates the view. ViewModel and the view are binded **through automated bindings**.

Model: - This section has business logic.





www.questpond.com

In MVVM the first hit comes to the view, view interacts with the viewmodel through **automated declarative bindings and commands**. ViewModel loads the model , has UI logic and updates the view through **automated bindings**.

What is the difference between MVP and MVVM ?

MVP does not have automated bindings and it submits itself completely to presenter. While in case of MVVM view talks to View Model through bindings.

In both cases the first hit comes to view.

What is a ViewModel?

Note:- We have ViewModel term in MVVM also and also generically ViewModel class is used. This answer is from the more generic perspective. ViewModel class is used generically in MVC, MVP and in many other architectures as well. The ViewModel class in MVVM has specialty of bindings.

ViewModel acts like a mediator between View and Model. ViewModel encapsulates the model and provides all things needed by view. ViewModel provides two things to view : -

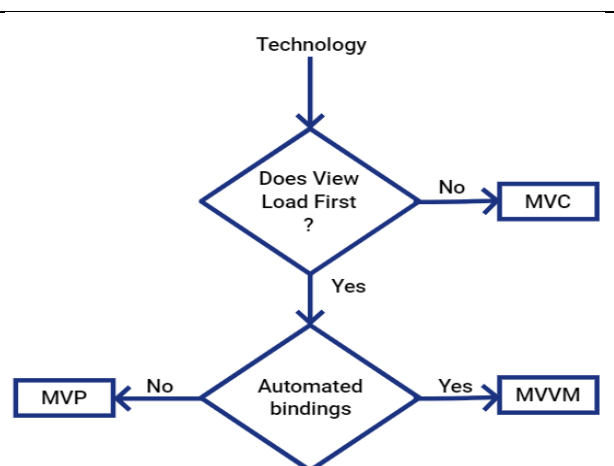
- Model Data:- examples - Customer code ,Customer name.
- View Data:- Enable , Disable , Color , Aesthetics and so on.

When to use what MVP / MVC / MVVM?

Putting it simple choice of these architecture pattern is not in the hand of the architect but **rather what kind of technical support that technology give**. So here is the decision road map.

- If your technology is loading the view first then you have only choice of MVP and MVVM.
- If your technology supports automated bindings then MVVM works well.
- If your technology does not load view first , like controller / component is loading the view then MVC suits well.

Note:- You can end up with combinational architecture also like Angular , component loads the view and also it has automated bindings. So it uses MVVM and MVC both.





www.questpond.com

MVC vs MVP vs MVVM?

	MVC	MVP	MVVM
Flow	Controller→Model/View	View→ Presenter →Model	View→VM→Model
	In MVC direct hit comes to controller while in case of MVP and MVVM direct hit comes to the view.		
Where to use?	ASP.NET MVC, Spring, Laravel, Angular	Webform, Winform, GWT, V AADIN Legacy.	WPF, Angular, Aurelia, Knockout, React
Automated Bindings	NA	NA	Declarative bindings.

Layered architecture vs Tiered?

Both layer and Tier emphasize SOC (Separation of concerns). So in both 3-layer and 3-tier we have 3 different units View, BL and Dal. View: - Takes care of Look and Feel, Positioning, Aesthetics etc. Business Logic: - This section as Business validations. Dal:- This section interacts with data store.	3 - Layer V ↔ BL ↔ DAL Deployment 3 - Tier VIEW ↔ Business Logic ↔ DAL HTTP REST TCP IP QUEUES
When we use the word 3-Layer :- We are discussing about how the project is organized logically. When we use the word 3-tier :- We are emphasizing that these sections are deployed in different machines/hardware. So in case of 3 tier the complexity increases as we need to now think on network communication (HTTP,TCP etc) and so on between these tiers.	



www.questpond.com

What is SOLID?

What is the full form of SOLID?

What is the goal of SOLID ?

Question :- Explain SRP with a example ?

Question :- What is the benefit of SRP ?

Question :- Explain OCP with a example ?

Question :- What is the benefit of OCP ?

Question :- Can you explain LISKOV Principle and its violation ?

Question :- How can we fix LISKOV Problem ?

Question :- Explain Interface Segregation Principle?

Question :- Is there a connection between LISKOV and ISP ?

Question :- Define dependency inversion?

Question :- What is higher level module and lower level module ?

Question :- How does dependency inversion benefit , show with an example ?

Question :- Will only Dependency inversion solve decoupling problem ?

Question :- Why do developers move object creation outside high level module?

Question :- Explain IOC (Inversion of control) ?

Question :- Explain Dependency Injection with an example?



www.questpond.com

Question :- Is SOLID, IOC and DI design pattern or principle?

Question :- Is only SOLID Enough for good code/architecture ?

2 or 3 Lessons on Design pattern

1 o 2 lecture on DDD and Microservice architecture ?

Requirement to Architecture how did you go about it

Some lectures covering Scenario based questions ?

How do you about writing a technical document?

Which UML diagrams have you used in your tech document

List of design patterns and Architecture patterns

GOF design patterns

Creational design patterns

Problems and solutions around object creation issues.



www.questpond.com

Abstract Factory - Creates an instance of several families of classes

Builder - Separates object construction from its representation

Factory Method - Creates an instance of several derived classes

Object Pool - Avoid expensive acquisition and release of resources by recycling objects that are no longer in use

Prototype - A fully initialized instance to be copied or cloned

Singleton - A class of which only a single instance can exist

Structural design patterns

Solving concerns around class structure and object composition.

Adapter - Match interfaces of different classes

Bridge - Separates an object's interface from its implementation

Composite - A tree structure of simple and composite objects

Decorator - Add responsibilities to objects dynamically

Facade - A single class that represents an entire subsystem

Flyweight - A fine-grained instance used for efficient sharing

Private Class Data - Restricts accessor/mutator access

Proxy - An object representing another object

Behavioral design patterns

Problems and solutions around Communication between objects.

Chain of responsibility - A way of passing a request between a chain of objects

Command - Encapsulate a command request as an object

Interpreter - A way to include language elements in a program

Iterator - Sequentially access the elements of a collection

Mediator - Defines simplified communication between classes

Memento - Capture and restore an object's internal state

Null Object - Designed to act as a default value of an object

Observer - A way of notifying change to a number of classes

State - Alter an object's behavior when its state changes

Strategy - Encapsulates an algorithm inside a class

Template method - Defer the exact steps of an algorithm to a subclass

Visitor - Defines a new operation to a class without change



www.questpond.com

Non-GOF Patterns

Which UML diagrams have you used in your tech document

High level vs Low level

Why Dependency injection ?

Factory pattern scenarios ?

Which Enterprise architecture patterns have you used?

What is SOLID?

Is DDD same like OOP?

Entity VS Value VS Service classes VS DTO ?

What are the core concepts of DDD?

Explain the difference between Bounded Contexts and Aggregates.

How do you identify Bounded Contexts in a system?

Describe your experience using any DDD tactical patterns (e.g., Repository, Factory).

How do you handle complex domain logic in a DDD architecture?

What was your thesis of dividing your Microservice project ?

If we use entity framework do we need repository pattern ?

Why do we need Event sourcing ?

How did you go about securing your application ?

Domain-Driven Design (DDD):

- What are the core concepts of DDD?
- Explain the difference between Bounded Contexts and Aggregates.
- How do you identify Bounded Contexts in a system?
- Describe your experience using any DDD tactical patterns (e.g., Repository, Factory).
- How do you handle complex domain logic in a DDD architecture?



www.questpond.com

- How to ensure continuous changes to existing system doesn't break the system. Basically something around TDD or testing your piece of code which has dependencies (Mocks/Fakes/Stubs etc.) and how best to approach testing the Frontend/ Backend systems

Asynchronous Programming using async and await

- How to secure Web API ,
- Clean architecture
- vertical slicing Architecture
- CQRS
- Exposure on Message queues such as RabbitMQ, MSMQ , Kafka etc
- Have any exposure to Polly library
- Options pattern
- Background Service or Hosted Service
- Middleware in NET Core and how to communicate between middleware
- Communication Protocols
- Containerization and Orchestration
- Data Management
- Event-Driven Architecture
- Security
- Logging and Monitoring
- Deployment Strategies
- Testing
- Documentation
- Scalability
- Api gateways
- Rabbitmq
- Architecture design vertical and horizontal